WasteZero: Smart Waste Pickup & Recycling Platform

Infosys Springboard Internship 7.0 program.

Complete Internship Project Documentation

MEAN / Angular Stack Full-Stack Development Internship
Milestones 1 – 4 | Weeks 1 – 8

Prepared by the WasteZero Development Team
Dhruv • Vaishnavi Shrivastava • Apoorva J • Aruneshwar

<u>Table of Contents</u>

SL. No.	Title	Page No.
1	**INTRODUCTION**	1
2	**PROJECT OVERVIEW**	2
3	**OBJECTIVES & SCOPE**	3
4	**REQUIREMENTS & TECHNOLOGIES USED**	4
5	**SYSTEM ARCHITECTURE**	5
6	**DATABASE DESIGN**	7
7	**MODULE IMPLEMENTATION**	9
	7.1 User Management	9
	7.2 Opportunity Management	9
	7.3 Matching & Communication	10
	7.4 Administration & Reporting	11
8	**MILESTONE-WISE IMPLEMENTATION**	13
	8.1 Milestone 1 – User Management	13
	8.2 Milestone 2 – Opportunity Management	13
	8.3 Milestone 3 – Matching & Communication	14
	8.4 Milestone 4 – Administration & Reporting	15
9	**USER INTERFACE & SCREENSHOTS**	16
10	**API DOCUMENTATION**	22
11	**TESTING & VALIDATION**	25
12	**DEPLOYMENT**	27
13	**CHALLENGES & SOLUTIONS**	28
14	**TEAM MEMBERS & CONTRIBUTIONS**	29
15	**FUTURE ENHANCEMENTS**	33
16	**CONCLUSION**	33
17	**REFERENCES**	34

1. INTRODUCTION

Improper waste disposal and inefficient coordination between volunteers, non-governmental organisations (NGOs), and waste-collection agents remain persistent challenges in community-driven environmental initiatives. Individuals who wish to responsibly dispose of recyclable and non-recyclable waste often lack a structured platform to schedule pickups, while NGOs and volunteer organisations struggle to reach the right audience for their waste-management and recycling drives. WasteZero was conceived as a digital solution to bridge this gap.

This document presents the complete internship project documentation for WasteZero, a Smart Waste Pickup & Recycling Platform developed using the Angular framework for the frontend and a Node.js / Express.js / MongoDB stack for the backend. The documentation captures the project's objectives, architecture, database design, module-wise and milestone-wise implementation details, user-interface walkthroughs, API reference, testing strategy, deployment approach, challenges encountered during development, and the individual contributions made by each member of the development team.

The project was executed over an eight-week internship period, structured into four two-week milestones. Each milestone incrementally built upon the previous one — beginning with core user management and authentication, followed by opportunity management, real-time matching and communication, and culminating in a comprehensive administration, moderation, and reporting layer. This staged approach allowed the team to validate functionality early, integrate frontend and backend work continuously, and deliver a fully functional, production-representative platform by the end of the internship.

This documentation is intended to serve as a durable technical and academic record of the WasteZero project — useful both as an internship deliverable and as a reference for anyone extending or maintaining the platform in the future.

2. PROJECT OVERVIEW

WasteZero is a full-stack web application that digitises the end-to-end lifecycle of community waste management — from a volunteer scheduling a pickup, to an NGO accepting and completing it, to an administrator monitoring platform-wide activity through analytics and downloadable reports. The platform supports three primary user roles — Volunteer, NGO, and Admin — each with a distinct, role-based dashboard and permitted set of actions.

2.1 Project Statement

WasteZero is a digital platform to help users schedule waste pickups, categorise recyclables, and promote responsible waste management. Pickup agents (NGOs) are assigned intelligently based on location and waste-type compatibility, and the platform surfaces volunteering opportunities that are matched to a volunteer's skills and location.

2.2 Core Outcomes

- Users can register, log in, and schedule waste pickups and Opportunities through a guided, multi-step workflow.
- Waste is categorised into standard types — plastic, paper, glass, e-waste, and organic — at the point of pickup and analytics.
- NGOs (agents) are notified of relevant pickups and can accept or reject them dynamically, based on location and waste-type matching.
- Volunteers can apply for the opportunities created by NGOs based on their skill set and locations
- Users receive real-time notifications and can track their personal environmental impact, including recycled weight and CO_2 saved.
- Administrators can monitor overall platform operations through a dedicated dashboard, moderation tools, and a multi-format reporting engine.

2.3 Functional Modules

The platform is organised into four functional modules, each of which maps directly onto one of the project's four development milestones:

Module	Description
Module A **User Management**	Registration, authentication, role-based access control, profile management, and account security (OTP, password recovery).
Module B **Opportunity Management**	Creation, discovery, and application workflow for NGO-posted volunteering / recycling opportunities.
Module C **Matching & Communication**	Waste-pickup scheduling, skill/location-based opportunity matching, real-time chat, and notifications.
Module D **Administration & Reporting**	Admin dashboard, user governance, content moderation, audit logging, analytics, and multi-format report generation.

3. OBJECTIVES & SCOPE

3.1 Objectives

- Design and implement a secure, role-based authentication system supporting Volunteer, NGO, and Admin roles.
- Provide NGOs with tools to create, manage, and track volunteering and recycling opportunities.
- Enable volunteers to discover relevant opportunities through an intelligent, skill- and location-based matching engine.
- Allow volunteers to schedule waste pickups and NGOs to accept, manage, and complete them.
- Deliver real-time, encrypted messaging and notifications between platform users.
- Provide administrators with governance tools — user suspension, role management, content moderation — backed by an append-only audit trail.
- Generate environmental-impact analytics (CO_2 saved, recycling breakdown) and exportable reports (CSV, Excel, PDF).
- Ensure the application is responsive, secure, and integrates a fully tested REST API layer with the Angular frontend.

3.2 Scope

The scope of the WasteZero platform, as delivered across the four milestones, includes:

- A complete Angular single-page application covering authentication, dashboards, opportunities, pickups, messaging, notifications, and administration.
- A Node.js/Express.js REST API backed by MongoDB, covering all functional modules described above.
- Real-time features implemented using Socket.IO for messaging and notifications.
- Security features including JWT-based authentication, OTP verification, AES-256-GCM message encryption, and role-based access control (RBAC).
- An administrative reporting engine capable of exporting Users, Pickups, Opportunities, and Full-Activity reports in CSV, XLSX, and PDF formats.

4. REQUIREMENTS & TECHNOLOGIES USED

4.1 Functional Requirements

- The system shall allow users to register with a role (Volunteer, NGO, or Admin) and verify their account via OTP.
- The system shall allow authenticated users to log in and receive a JWT for subsequent protected-API access.
- The system shall allow NGOs to create, edit, and delete volunteering opportunities with required skills, duration, and location.
- The system shall allow volunteers to search, filter, and apply to opportunities, and to withdraw applications.
- The system shall allow volunteers to schedule, edit, and delete waste-pickup requests.
- The system shall allow NGOs to view, accept, or reject pickup requests relevant to their location and capability.
- The system shall recommend opportunities and pickups to users using a matching algorithm based on skills, waste type, and location.
- The system shall provide real-time chat and notifications between Volunteers and NGOs.
- The system shall allow Admins to view, suspend, and manage users; moderate opportunities; and view an audit trail of administrative actions.
- The system shall generate downloadable analytical reports in CSV, Excel, and PDF formats.

4.2 Non-Functional Requirements

- Security: passwords must be hashed; sensitive data (messages, notifications) must be encrypted at rest; administrative endpoints must enforce RBAC.
- Performance: list-based APIs must support pagination; large report exports must stream data rather than loading entire datasets into memory.
- Usability: the UI must be responsive across desktop and mobile breakpoints and must support both light and dark themes.
- Reliability: administrative audit logs must be append-only and tamper-resistant.
- Maintainability: the codebase must use a consistent, standardised API response structure and modular service/controller architecture.

4.3 Technology Stack

Layer	Technology
Frontend Framework	Angular
Frontend Language	TypeScript, HTML5, CSS3
Backend Runtime	Node.js
Backend Framework	Express.js
Database	MongoDB (Mongoose ODM)
Real-Time Communication	Socket.IO (WebSockets)
Authentication	JSON Web Tokens (JWT), OTP-based verification
Data Encryption	AES-256-GCM (messages & notifications at rest)
API Testing	Postman
Report Generation	PDFKit / pdfkit-table, ExcelJS, Node.js CSV streaming
Version Control	Git
Project Style	REST API architecture with layered Controller → Service → Model design

5. SYSTEM ARCHITECTURE

WasteZero follows a classic three-tier, client–server architecture. The Angular single-page application (SPA) forms the presentation tier, communicating with a Node.js/Express.js REST API (the application tier) over HTTPS, which in turn persists and retrieves data from a MongoDB document store (the data tier). Real-time features — messaging, typing indicators, read receipts, and notifications — are layered on top of the REST API using Socket.IO, allowing the server to push events to connected clients without polling.

5.1 High-Level Architecture Flow

- Presentation Tier (Angular): role-based dashboards, reactive forms, route guards, and an HTTP interceptor layer that attaches JWTs to outgoing requests.
- Application Tier (Node.js / Express.js): REST controllers, business-logic services, authentication & RBAC middleware, validation middleware, rate limiters, and the Socket.IO event layer.
- Data Tier (MongoDB): collections for Users, Opportunities, Applications, Pickups, WasteStats, Messages, Notifications, and AdminLogs, accessed through Mongoose schemas.

5.2 Request Lifecycle

A typical protected API request flows through the following layered pipeline before reaching business logic:

1. Client Request — the Angular service issues an HTTP call with a bearer JWT attached by the HTTP interceptor.

2. Authentication Middleware — verifies the JWT and attaches the authenticated user's identity to the request context.

3. Authorization / RBAC Middleware — checks the user's role against the route's required permissions (e.g. requireAdmin).

4. Rate Limiting — sensitive or expensive operations (e.g. report generation) pass through a request-rate limiter.

5. Validation Middleware — verifies request payloads against expected schemas before reaching the controller.

6. Controller → Service → Model — the controller delegates to a service layer, which applies business rules and interacts with Mongoose models.

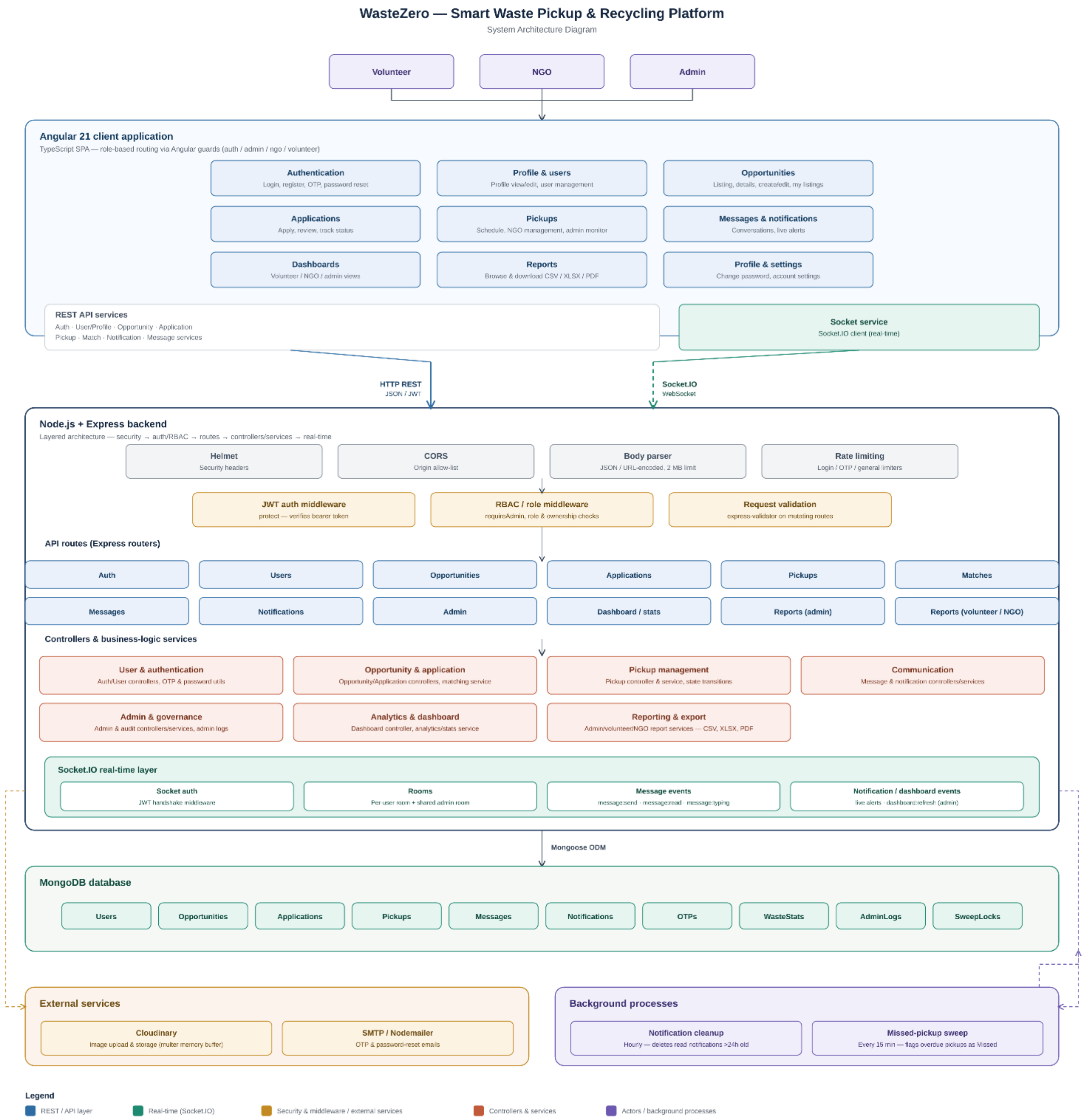
7. Response — a standardised response envelope (utils/apiResponse.js) is returned to the client.

5.3 Real-Time Architecture (Socket.IO)

Each authenticated user is placed into a personal Socket.IO room in the form user:<userId> upon connection. This room-based design allows the backend to target a specific user with messages, notifications, and read-receipt events — regardless of how many browser tabs or devices that user has open — without manually tracking individual socket IDs. Socket connections are authenticated using the same JWT issued at login, and administrative suspension immediately revokes a user's active socket session alongside their REST API access.

5.4 Security Architecture

Security is enforced in layers across the stack: password hashing at rest, JWT-based stateless authentication, OTP verification for registration and password recovery, AES-256-GCM encryption for message and notification content, MongoDB ObjectId and payload validation, role-based access control for administrative routes, and an append-only audit logging system that records every sensitive administrative action together with the acting admin, the affected resource, before/after values, IP address, user agent, and timestamp.



6. DATABASE DESIGN

WasteZero uses MongoDB as its primary data store, with Mongoose providing schema definition, validation, and relationship management through referenced ObjectIds. The schema was designed collaboratively, with the core entities established in Milestone 1 and 2, and extended in Milestones 3 and 4 to support pickups, waste statistics, messaging, notifications, and administrative logging.


WasteZero - MongoDB Entity Relationship Diagram
Smart Waste Pickup & Recycling Platform
SWEEPLOCKS
PK _id String
instanceId String
lockedAt Date
lockedUntil Date
OTPS
(schema not exposed in supplied data model)
MESSAGES
(schema not exposed in supplied data model)
LEGEND
PK = Primary Key
FK = Foreign Key / Reference
ObjectId = MongoDB ObjectId
—< = crow's foot (many side)
Dashed line = optional / polymorphic ref
APPLICATIONS
PK _id ObjectId
FK opportunity_id ObjectId → opportunities._id
FK volunteer_id ObjectId → users._id
status String
createdAt Date
updatedAt Date
ADMINLOGS
PK _id ObjectId
FK admin_id ObjectId → users._id
action String
target_type String
FK target_id ObjectId → polymorphic
details String
changes Object {before: Mixed, after: Mixed}
ip_address String
user_agent String
timestamp Date
createdAt Date
updatedAt Date
OPPORTUNITIES
PK _id ObjectId
title String
description String
date Date
duration String
location String
image String
imagePublicId String
required_skills [String]
status String
FK ngo_id ObjectId → users._id
isRemovedByAdmin Boolean
removedBy Mixed
removedAt Date
removalReason String
createdAt Date
updatedAt Date
USERS
PK _id ObjectId
username String
email String
password String
role String
bio String
skills [String]
wasteTypes [String]
locations Object {primary, secondary[]}
isVerified Boolean
isSuspended Boolean
suspendedAt Date
suspendedBy Mixed
suspensionReason Mixed
createdAt Date
updatedAt Date
PICKUPS
PK _id ObjectId
FK user_id ObjectId → users._id
FK agent_id Mixed → users._id
address Object {area, city}
wasteTypes [String]
preferredTimeSlot Object {start, end}
scheduledDate Date
status String
rescheduleCount Number
notes String
missedAt Date
missedCutoffAt Date
completedAt Mixed
createdAt Date
updatedAt Date
WASTESTATS
PK _id ObjectId
FK user_id ObjectId → users._id
FK ngo_id ObjectId → users._id
FK pickup_id ObjectId → pickups._id
category String
weight Double
co2 saved kg Double
date Date
createdAt Date
updatedAt Date
NOTIFICATIONS
PK _id ObjectId
FK user_id ObjectId → users._id
FK reference_id ObjectId → polymorphic
type String
message String
isRead Boolean
iv String
authTag String
readAt Date
createdAt Date
updatedAt Date
admin_id (N:1)
opportunity_id (N:1)
volunteer_id (N:1)
target_id (polymorphic, via target_type)
ngo_id (N:1)
reference_id (polymorphic, via type)
user_id (N:1)
agent_id (N:1, optional)
pickup_id (N:1)

7. MODULE IMPLEMENTATION

This section describes the functional design and implementation approach for each of WasteZero's four core modules, spanning both the Angular frontend and the Node.js/Express.js backend.

7.1 User Management

The User Management module forms the security and identity foundation of the platform. On the backend, it comprises the Authentication Controller and User Controller, exposing registration, login, OTP verification, password recovery/reset, change-password, and profile-update APIs, all backed by JWT-based protected-route middleware and Mongoose-integrated MongoDB persistence. Passwords are never stored in plaintext; they are hashed before persistence, and every subsequent login re-verifies the submitted credential against the stored hash.

Registration is guarded by an OTP-based email-verification flow: a user registers, an OTP is generated and sent, and the account is not considered verified until the OTP is confirmed. The same OTP infrastructure underpins the Forgot Password workflow, ensuring that a password can only be reset after a fresh OTP challenge has been completed — closing a common account-takeover vector.

On the frontend, the module includes a complete Login and Registration UI integrated with the backend authentication APIs, OTP-entry screens, a Change Password interface, a role-based dashboard shell that renders different navigation and widgets depending on whether the logged-in user is a Volunteer, NGO, or Admin, and a Profile Management screen for editing personal details such as name, location, skills, and bio.

Key Capabilities

- Role-based registration (Volunteer / NGO / Admin) with OTP email verification.
- JWT-based login and protected route access, enforced consistently across both REST and Socket.IO layers.
- OTP-secured Forgot Password and Reset Password workflow.
- Change Password with current-password verification.
- Role-aware dashboard shell and full profile-editing UI.

7.2 Opportunity Management

The Opportunity Management module allows NGOs to publish volunteering and recycling opportunities and allows volunteers to discover and apply to them. The backend exposes a complete CRUD API — Create, Get All, Get by ID, Update, Delete, My Opportunities, Search, and Filter — with ownership-aware authorization ensuring that only the NGO that created an opportunity (or an Admin) can modify or remove it.

An Application sub-module manages the relationship between volunteers and opportunities: applying, retrieving applications (by opportunity or by the authenticated user), viewing an individual application, updating an application's status during NGO review, and withdrawing an application. A shared query-builder/pagination utility standardises list behaviour across both the Opportunity and Application APIs.

On the frontend, the module was first scaffolded with an initial Opportunity List and Detail page, a service layer for backend communication, and baseline routing. It was then substantially extended into a full-featured module: search and filtering, a reusable pagination component, a shared Create/Edit form with validation, a Delete Confirmation dialog, a My Opportunities dashboard for NGOs, the complete Apply / My Applications / application-status workflow for volunteers, and NGO/Admin route guards restricting write access to authorised roles.

Key Capabilities

- NGO-authored opportunity creation, editing, and deletion with ownership validation.
- Searchable, filterable, paginated opportunity listing with loading, error, and empty states.
- Full volunteer application lifecycle: apply, track status, withdraw.
- My Opportunities dashboard for NGOs to manage their own postings.
- Role-based route guards restricting opportunity management to NGO and Admin users.

7.3 Matching & Communication

This module covers three tightly related capabilities: waste-pickup scheduling and management, intelligent matching, and real-time messaging/notifications.

7.3.1 Pickup Management

Volunteers can create, edit, and delete waste-pickup requests specifying address, city, waste type(s), and preferred time. NGOs see available pickup requests filtered by their location and supported waste types, and can accept or reject them; accepted pickups appear in a dedicated My Assigned Pickups view. Administrators have read-level visibility across all pickups platform-wide, including the ability to override a pickup's status to resolve exceptional states such as stuck or orphaned assignments — an action that is recorded in the audit log as PICKUP_STATUS_OVERRIDE.

7.3.2 Matching Algorithm

The matching engine compares a volunteer's stored skills and location against each opportunity's required skills and location to surface the most relevant opportunities directly on the volunteer dashboard. A parallel matching process pairs pickup requests with NGOs based on the NGO's location and the waste type(s) it services, returning a ranked list of top-matching results. Both matching pathways are integrated with the notification system, so users are proactively alerted the moment a suitable match becomes available.

7.3.3 Real-Time Messaging

Real-time communication is implemented with Socket.IO. Core events include message:send (validates sender/receiver, generates a deterministic conversation ID, encrypts and persists the message, and emits it to the recipient), message:new (delivers incoming messages in real time), message:typing (live typing indicators), and message:read (read-receipt propagation with a stored readAt timestamp). All message content is encrypted at rest using AES-256-GCM and transparently decrypted when a conversation is requested by an authorised participant.

The frontend chat experience includes username/user search for starting new conversations, conversation search, chat history rendering, live typing indicators, sent/read message-status badges, and direct "Contact NGO" / "Message applicant" entry points embedded directly in the Opportunity and Application views to minimise the steps required to start a conversation.

7.3.4 Notifications

Notifications are separated into General (platform/system events) and Text (message-related) categories for clarity, each with unread indicators. Volunteers are notified when a new opportunity matches their profile; NGOs are notified when a new pickup request matches their location and waste-type capability. Notification content is encrypted using the same AES-256-GCM scheme used for messages.

7.4 Administration & Reporting

The Administration & Reporting module, delivered in Milestone 4, gives administrators controlled, auditable authority over the platform. It spans user governance, content moderation, security enforcement, and a multi-format analytics/reporting engine.

7.4.1 User Governance

Administrators can list, search, and filter users by role and suspension status. A suspension action records the reason, timestamp, and acting admin, and is treated as a security event: it immediately revokes the affected user's active Socket.IO session and blocks further protected-API access, not merely flipping a database field. Role management allows an admin to change a user's role between volunteer, ngo, and admin, with safeguards against self-demotion and against removing the last remaining admin account.

7.4.2 Content Moderation

Inappropriate opportunities can be removed by an administrator using soft deletion — the record is flagged as removed (with reason, timestamp, and admin) rather than being permanently deleted, so it disappears from public listings and can no longer receive new applications while preserving historical application data. Where supported, a moderated opportunity can later be restored, clearing the moderation metadata.

7.4.3 Audit Logging

Every sensitive administrative mutation — suspension, role change, opportunity removal/restoration, pickup status override, and report download — is written to an append-only AdminLogs collection capturing who performed the action, what the action was, which resource was affected, the before/after values, the request's IP address and user agent, and a timestamp. No API exists to modify or delete an existing audit entry.

7.4.4 Analytics & Reporting

An Admin Dashboard Analytics endpoint aggregates platform-wide KPIs — user counts by role, pickup totals and completion state, active opportunities, application counts, and month-over-month growth percentages. A parallel personal-dashboard endpoint surfaces the same class of metrics scoped to an individual user's own activity and environmental impact (pickups, recycled items, CO_2 saved, volunteer hours).

A dedicated recycling-breakdown endpoint returns a category-level analysis (plastic, paper, glass, e-waste, organic) of weight, percentage share, and CO_2 saved for a given month, computed using MongoDB aggregation pipelines rather than in-application post-processing, for scalability. A CO_2-calculator utility centralises the weight-to-CO_2-savings conversion so the calculation stays consistent across the platform.

The reporting engine supports four report types — Users, Pickups, Opportunities, and Full Platform Activity — each exportable as CSV, XLSX, or PDF, with date-range filtering and validation. To remain performant on large datasets, report generation streams data from a MongoDB cursor through a transform pipeline directly to the HTTP response rather than materialising the entire dataset in memory, for all three export formats. Report requests pass through a dedicated rate limiter given their computational cost, and every successful report download is recorded in the audit log as REPORT_DOWNLOADED.

7.4.5 Administration Frontend

The Angular frontend provides a dedicated Admin Users page (table, search, role/status filters, pagination, suspend/activate actions with confirmation modals), an Admin Logs page (filterable, paginated view of the audit trail), and a Reports page (report-type selection, date filtering, format selection, and download handling with loading/error states). All administrative routes are protected by frontend route guards restricting access to Admin users only, mirroring the backend's RBAC enforcement.

8. MILESTONE-WISE IMPLEMENTATION

The WasteZero project was delivered across four two-week milestones. This section presents the objectives, delivered output, and implementation highlights for each milestone in the order they were executed, reflecting how the platform was incrementally built up from a foundational authentication layer to a fully governed, analytics-driven system.

8.1 Milestone 1 – User Management (Weeks 1–2)

Objectives

- Develop user registration and authentication with role-based access control.
- Implement user profile management, including edit functionality.
- Configure and integrate the backend database to securely store user information.

Delivered Output

- Functional Login and Registration interfaces integrated end-to-end with backend APIs.
- A role-based user dashboard tailored to each account type.

Implementation Highlights

- Backend: Authentication Controller, User Controller, Registration/Login/Change-Password/Update-Profile APIs, JWT-based protected routes, MongoDB integration via Mongoose, and password hashing (Vaishnavi).

- Backend: OTP-based registration verification, Forgot Password, and Reset Password workflows (Apoorva J).

- Frontend: complete Login/Registration UI, OTP entry pages, role-based dashboard, and profile editing screens integrated with live backend APIs; CORS and routing issues identified and resolved during integration (Dhruv).

- Frontend: after a team change, ownership of the Change Password UI and its backend integration was transferred to and completed by Dhruv.

8.2 Milestone 2 – Opportunity Management (Weeks 3–4)

Objectives

- Enable NGOs to create, update, and delete volunteering opportunities.
- Capture essential opportunity details — required skills, duration, and description.
- Design and implement a database schema to manage opportunity records.

Delivered Output

- Opportunity creation and editing form.
- Opportunity listing interface for end users.

Implementation Highlights

- Backend: Vaishnavi designed the Opportunity schema and finalised the API contract, then implemented the complete Opportunity CRUD API, ownership-based authorization, search/filter endpoints, and the shared apiResponse.js response-standardisation utility; all endpoints were tested via Postman.

- Backend: Apoorva J designed the Application schema and implemented the full Application API — apply, get applications, get by ID, update status, withdraw, and My Applications — along with a shared query-builder/pagination utility, and tested it via Postman.

- Frontend: Aruneshwar established the initial Opportunity module — List and Detail pages, navigation, an Opportunity service with basic CRUD methods, and initial routing — laying the foundation for the module.

- Frontend: Dhruv substantially extended this foundation with search, filtering, a reusable pagination component, the shared Create/Edit form with validation, delete-confirmation dialog, a My Opportunities dashboard, the complete Applications workflow, NGO/Admin route guards, and final integration with the finalised backend APIs.

8.3 Milestone 3 – Matching & Communication (Weeks 5–6)

Objectives

- Implement a matching algorithm connecting volunteers to opportunities based on waste type and geographic location.
- Develop real-time messaging using WebSockets for seamless communication between users.
- Integrate a notification system to alert users of new matches and incoming messages.

Delivered Output

- Interactive chat interface for real-time communication.
- Match-suggestion dashboard displaying relevant opportunities.

Implementation Highlights

- Backend: Vaishnavi designed and implemented the Socket.IO messaging architecture — message:send/new/typing/read events, AES-256-GCM encryption for messages and notifications, deterministic conversation-ID generation, authenticated per-user Socket.IO rooms, the Notification Service (dispatch, listForUser, markRead), and the complete backend test suite covering socket auth, encryption/decryption, and persistence.

- Backend: Apoorva J implemented the complete Pickup Management backend (volunteer create/edit/delete, NGO accept/reject, admin visibility), the skill-and-location-based Opportunity Matching algorithm, the location-and-waste-type Pickup Matching algorithm, and contributed the REST API layer supporting the Messaging and Notification modules alongside the Socket.IO implementation.

- Frontend: Dhruv implemented the complete frontend for this milestone — Schedule Pickup and Pickup History for volunteers; Available Pickups and My Assigned Pickups for NGOs; Admin Pickup Management; Opportunity Matching results on the volunteer dashboard; and the full messaging UI (user/conversation search, chat history, typing indicators, message-status badges, and direct chat entry points from opportunities and applications), plus the categorised General/Text notification system with unread indicators.

8.4 Milestone 4 – Administration & Reporting (Weeks 7–8)

Objectives

- Develop an admin dashboard to monitor user activity and platform engagement.
- Generate analytical reports on active users, posted opportunities, and volunteer responses.
- Implement administrative controls to suspend users or remove posts when necessary.
- Complete testing and final project reporting.

Delivered Output

- Comprehensive admin panel with activity-oversight tools.
- Downloadable reports for platform performance.

Implementation Highlights

- Backend: Vaishnavi took ownership of platform governance — admin user listing/search/filtering, suspension and unsuspension with session/socket revocation, role management with last-admin protection, opportunity moderation and restoration, the requireAdmin RBAC middleware, the append-only Admin Audit Log system and API, and administrative rate limiting and input validation.

- Backend: Apoorva J delivered the Analytics & Reporting backend — admin dashboard statistics, personal dashboard metrics, recycling-breakdown analytics, WasteStats generation and the CO_2-calculator utility, administrative pickup analytics with status override, and the full Report Generation System (CSV, Excel/XLSX, and PDF exporters) with streaming, filtering, validation, rate limiting, and audit integration.

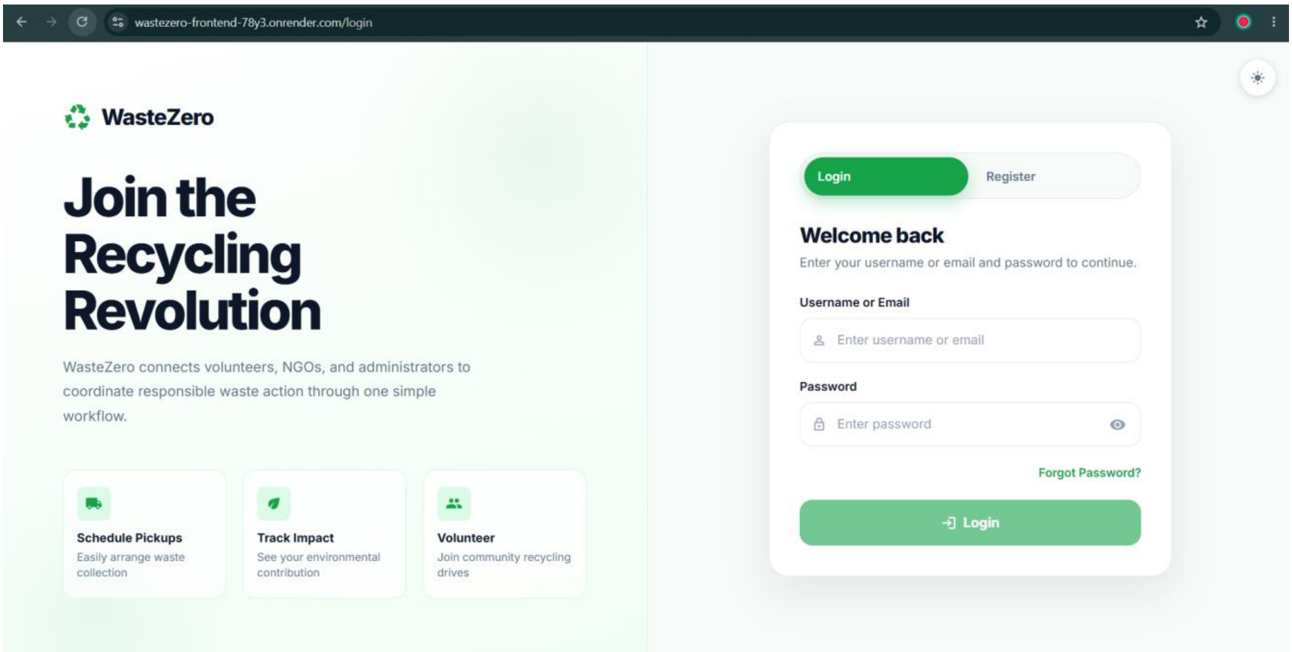
- Frontend: Dhruv completed the Admin Dashboard, Admin Controls Panel, and the Reports/Dashboard interface, with role-aware behaviour, responsive layouts, and final UI refinement across the administrative frontend.

- Frontend: Aruneshwar built the dedicated Admin Users page, Admin Logs page, and Reports interface (report-type selection, date/format filtering, and download handling), plus the /admin/* navigation structure and frontend route protection restricting these pages to Admin users; mock data was used temporarily where backend report APIs were still in progress, to validate the frontend workflow independently.

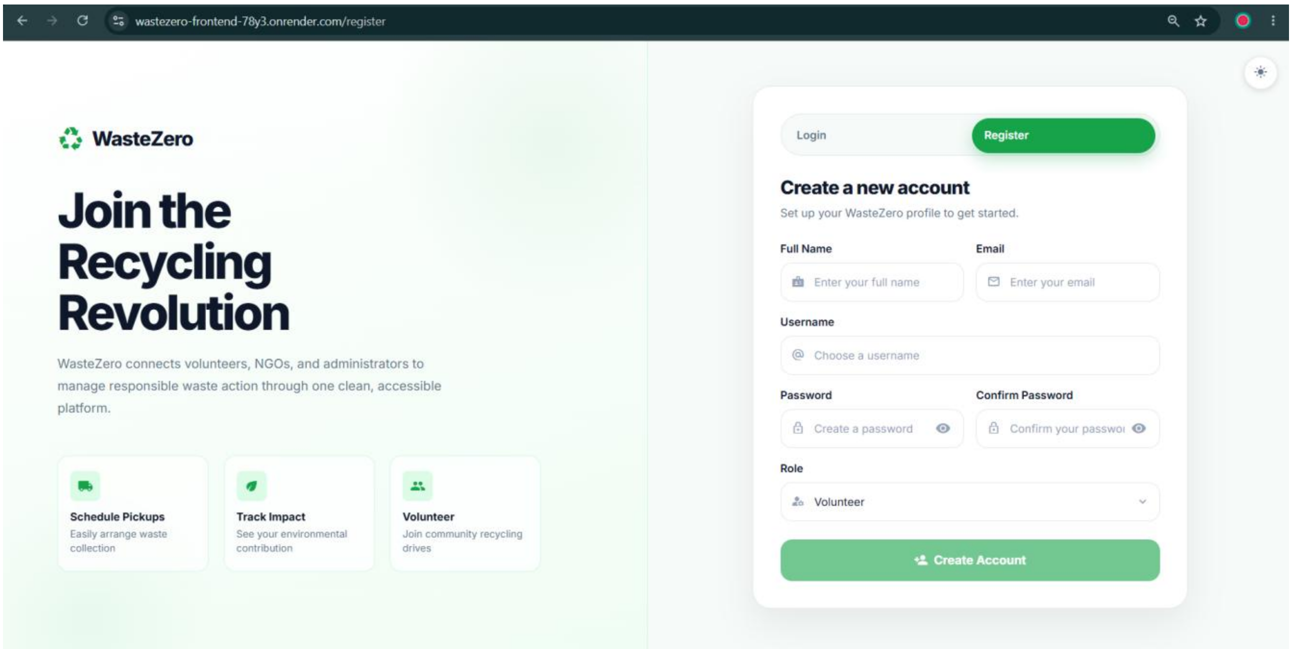
9. USER INTERFACE & SCREENSHOTS

The following screenshots illustrate the WasteZero Angular frontend across its major milestones, captured from the interactive design mockups used to guide implementation. The interface supports both light and dark themes and is fully responsive across desktop and mobile breakpoints.

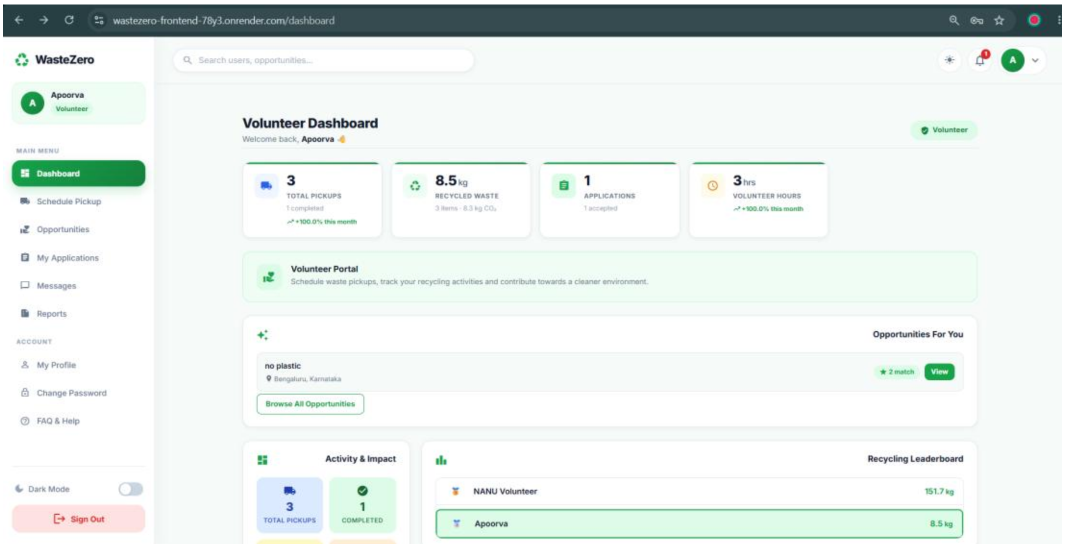
Login Page



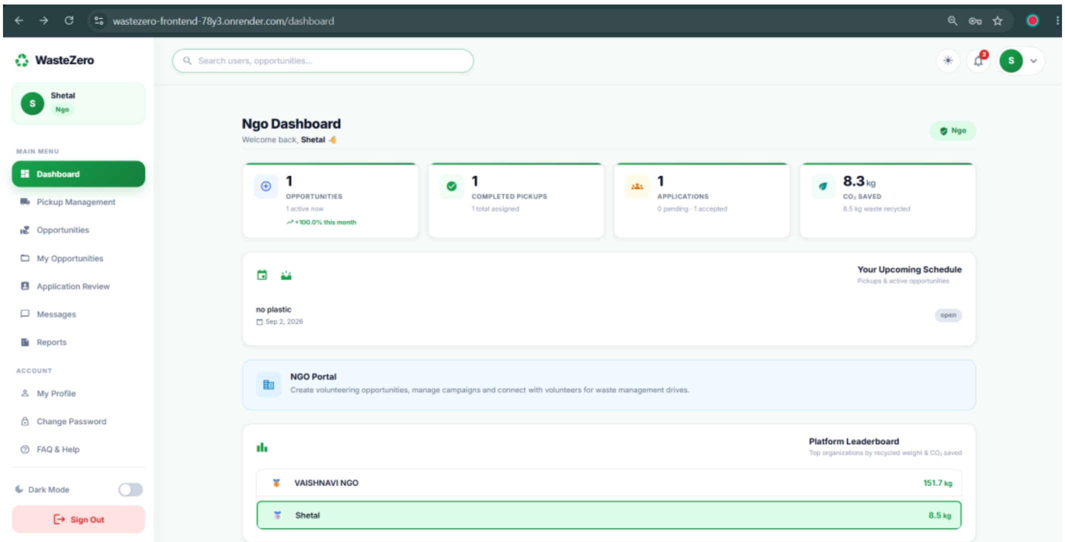
Registration Page



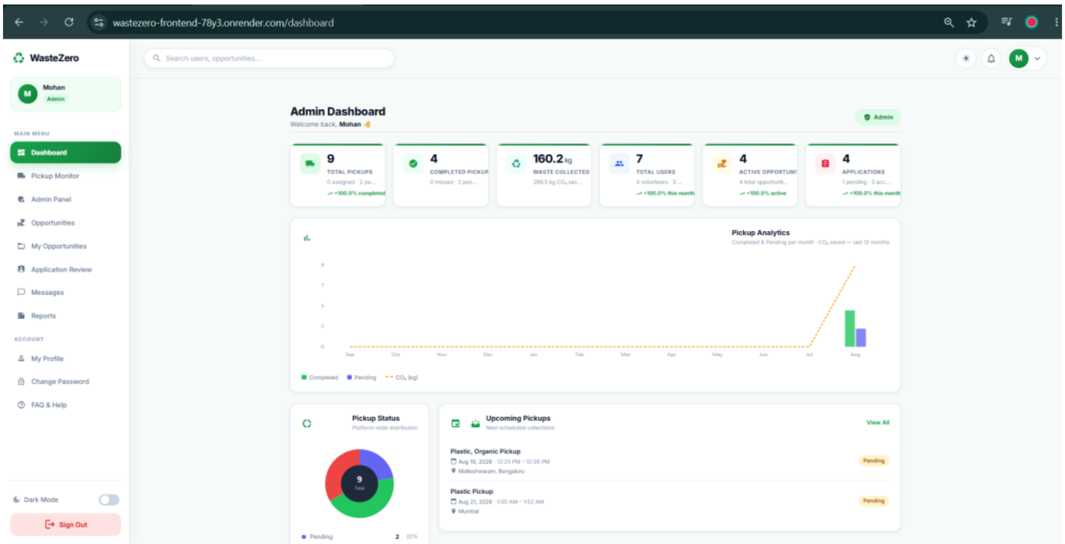
Volunteer Dashboard



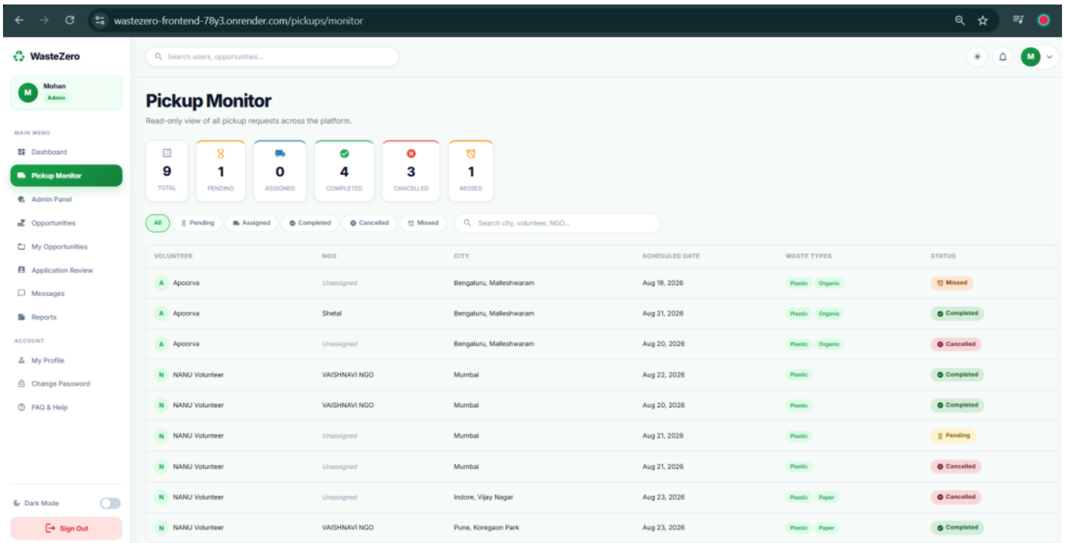
Ngo Dashboard



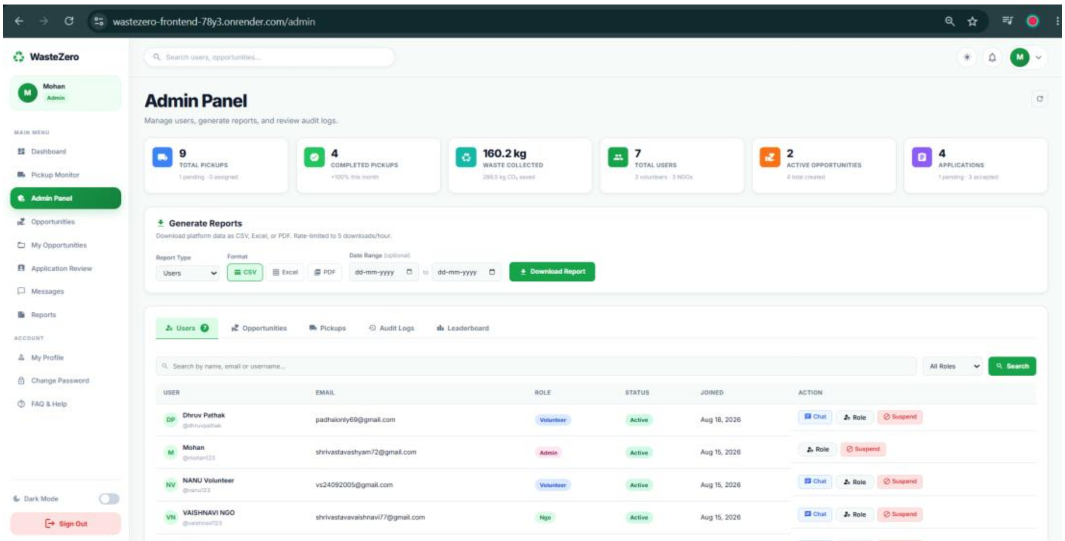
Admin Dashboard



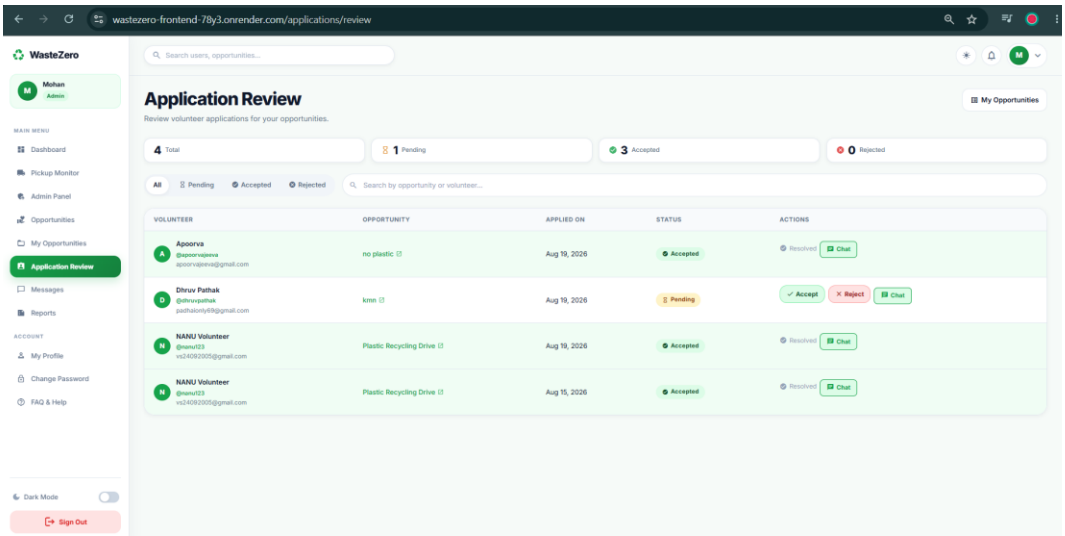
Admin Pickup Monitor



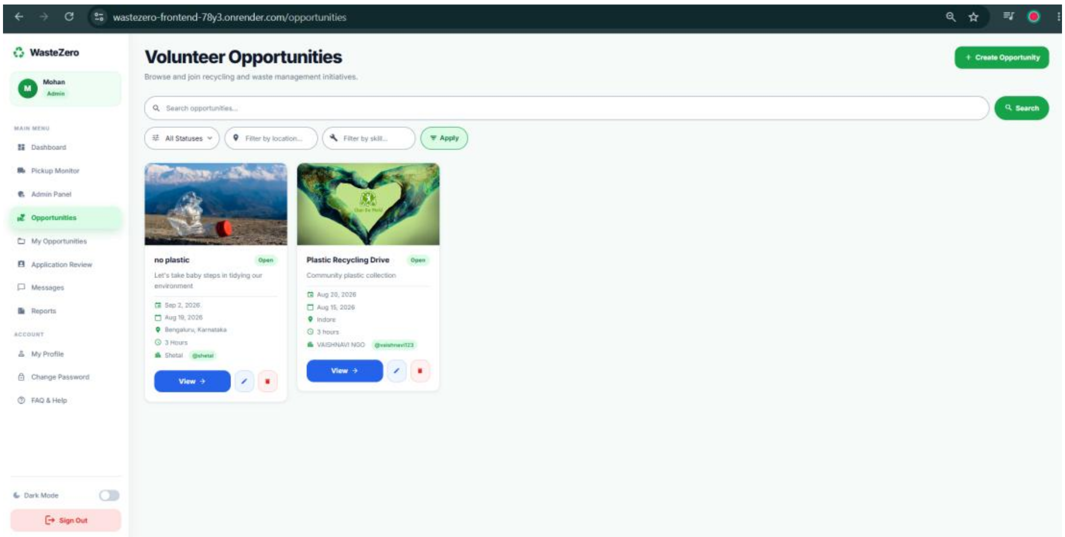
Admin Panel



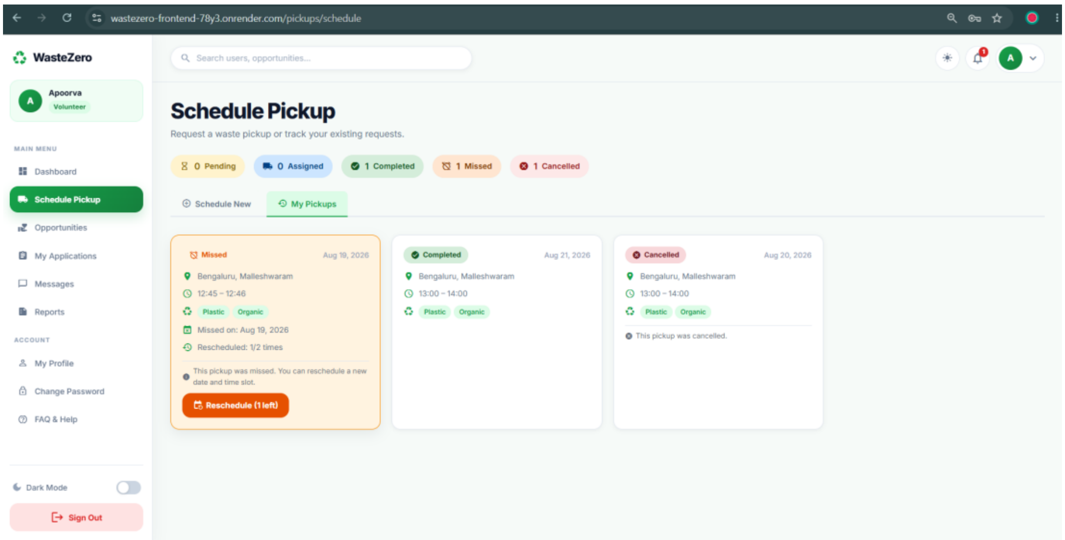
Applications review



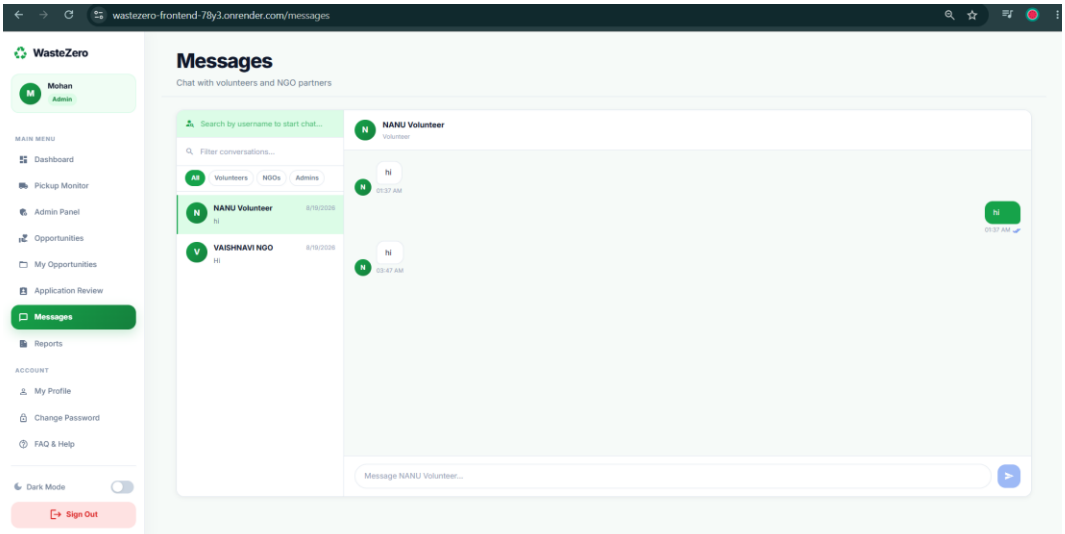
Opportunities Creation Page



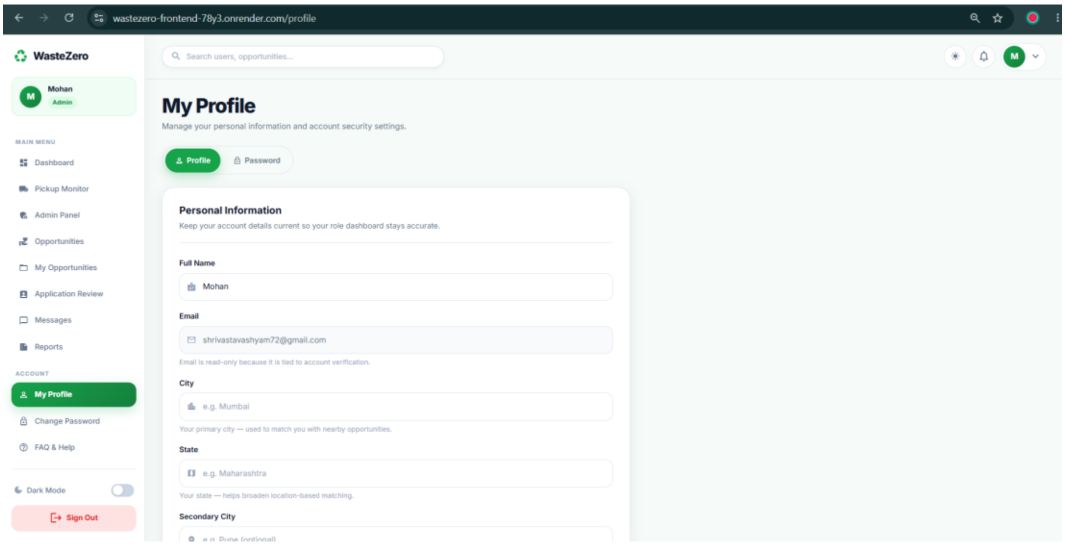
Pickup Creation Page



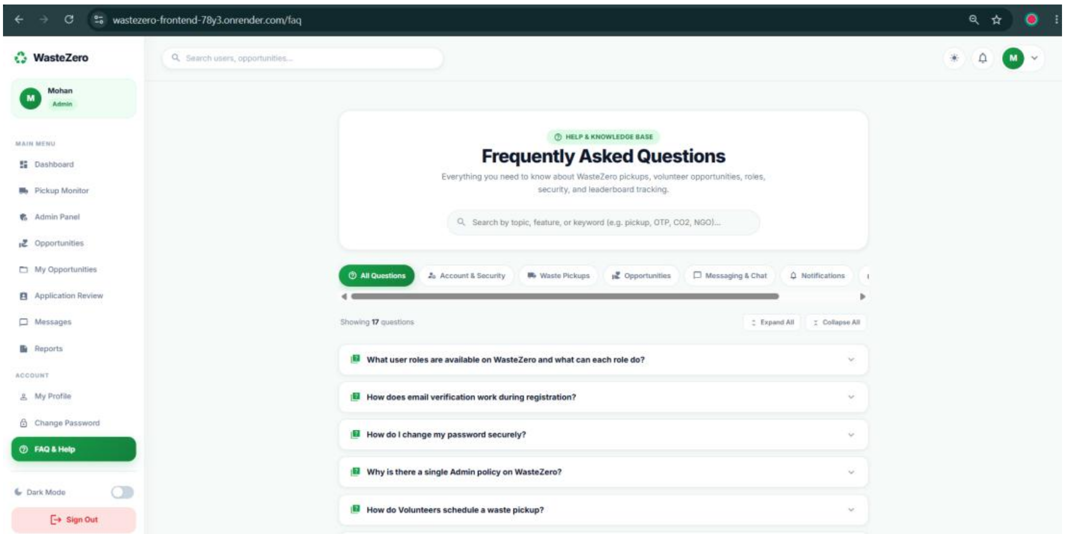
Messaging Interface



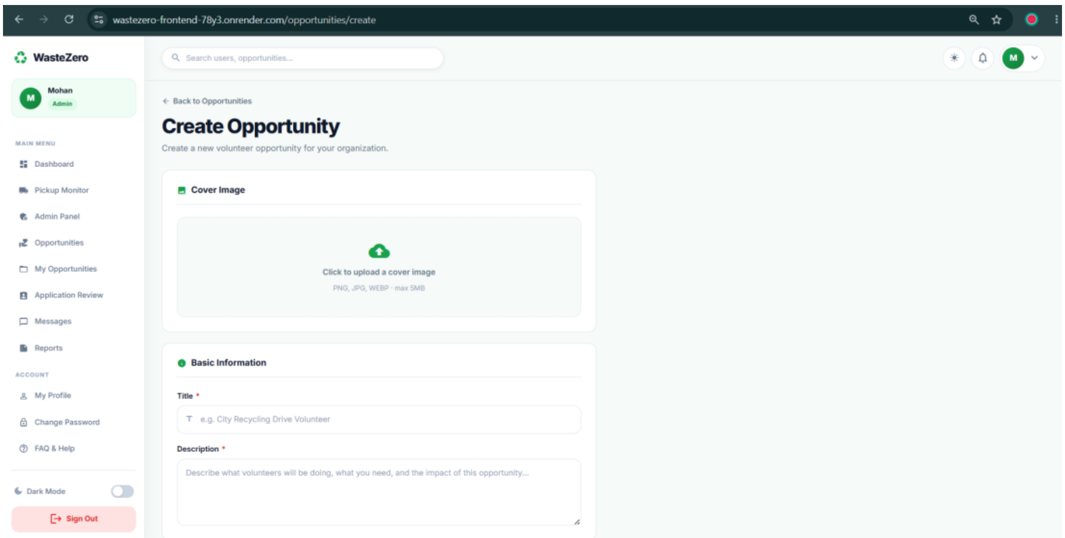
Profile Page



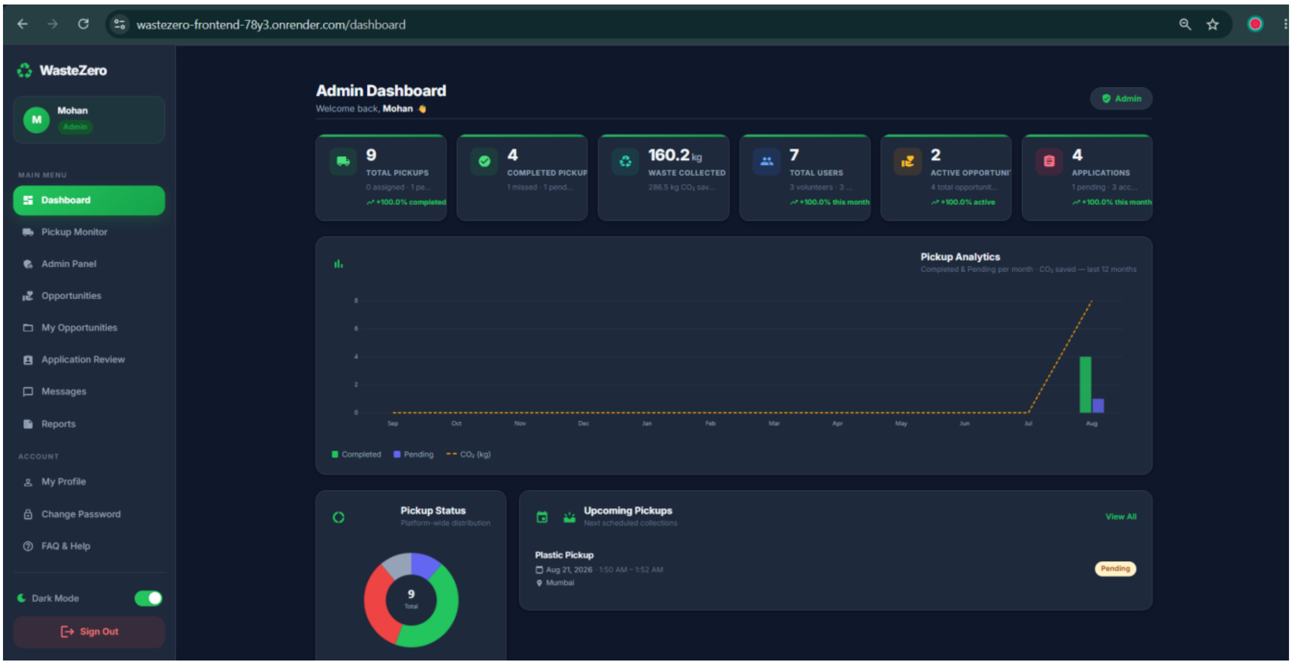
FAQ & Help Page



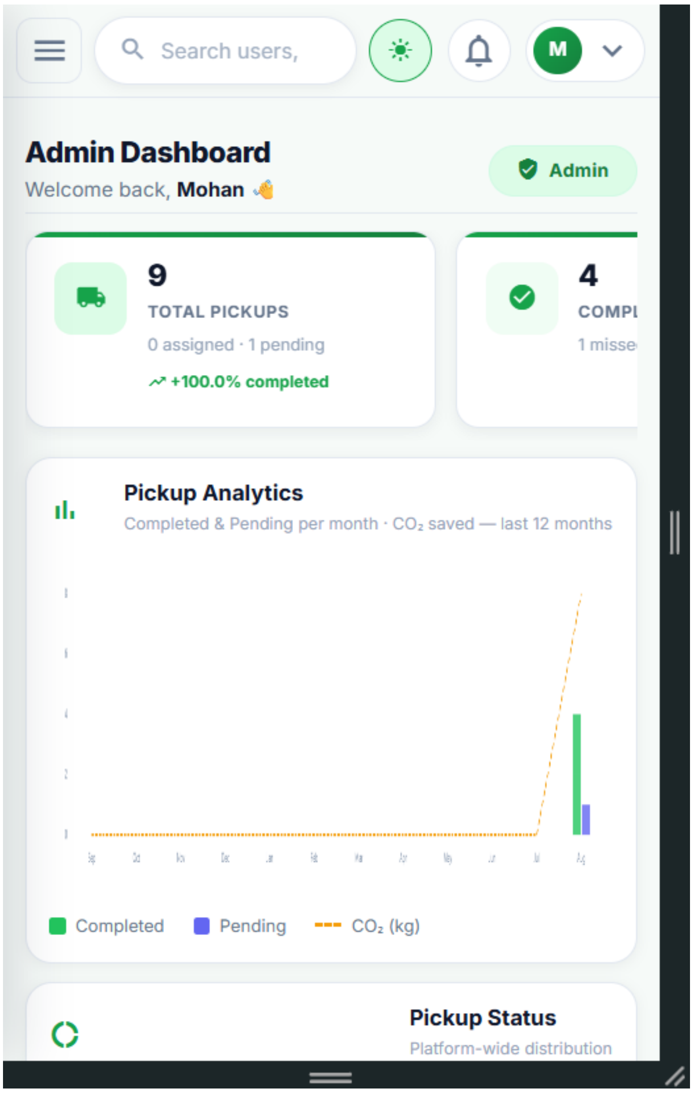
Create Opportunity Page



Dark Mode



Mobile View



10. API DOCUMENTATION

WasteZero exposes a versioned REST API under /api/v1. All protected endpoints require a valid JWT bearer token; administrative endpoints additionally require the admin role, enforced through the requireAdmin middleware. All list endpoints support standard pagination via page and limit query parameters, and all responses follow a consistent envelope produced by the shared apiResponse.js utility.

10.1 Authentication and User APIs

Method	Endpoint	Access	Description
POST	/register	Public	Register new Volunteer or NGO account
POST	/admin/setup	Secret Gated	Bootstrap primary system administrator
POST	/login	Public	Authenticate user & receive JWT token
POST	/verify-otp	Public	Validate email verification OTP code
POST	/resend-otp	Public	Request a fresh OTP token
POST	/forgot-password	Public	Request password reset token
POST	/reset-password	Public	Submit new password with verification token
GET	/profile	Authenticated	Retrieve authenticated user profile
PUT	/profile	Authenticated	Update user bio, skills, location, and avatar
GET	/search	Authenticated	Cross-role user search directory
GET	/settings	Authenticated	Retrieve user preferences
PUT	/settings	Authenticated	Update theme, email, and notification settings
POST	/change-password /send-otp	Authenticated	Request OTP to change password
PUT	/change-password /verify-otp	Authenticated	Commit password update with OTP

10.2 Administration & Audit APIs

Method	Endpoint	Access	Description
GET	/users	Admin	Paginated user management directory
GET	/users/:id	Admin	Detailed user profile and audit info
PATCH	/users/:id/suspend	Admin	Suspend or reactivate user account
PATCH	/users/:id/role	Admin	Change user role (volunteer, ngo, admin)
DELETE	/opportunities/:id	Admin	Moderation soft-delete of opportunity
PATCH	/opportunities/:id/restore	Admin	Restore moderated opportunity
GET	/logs	Admin	Paginated append-only audit trail logs

10.3 Opportunity & Application APIs

Method	Endpoint	Access	Description
GET	/opportunities	Authenticated	Browse public volunteering drives
POST	/opportunities	NGO / Admin	Create new opportunity with image upload
GET	/opportunities/:id	Authenticated	Get opportunity details
PUT	/opportunities/:id	Owner / Admin	Update opportunity details
DELETE	/opportunities/:id	Owner / Admin	Delete opportunity
GET	/matches/suggestions	Volunteer	Get ranked recommendations by skills & geo
POST	/applications	Volunteer	Apply for a volunteering opportunity
GET	/applications/my-applications	Volunteer	List submitted volunteer applications
GET	/applications	NGO / Admin	List applications for hosted opportunities
PUT	/applications/:id	NGO / Admin	Accept or reject application
DELETE	/applications/:id	Volunteer	Withdraw pending application

10.4 Messaging & Notification APIs

Method	Endpoint	Access	Description
GET	/messages/conversations	Authenticated	Get user conversation threads
GET	/messages?with=<userId>	Authenticated	Retrieve decrypted 1-on-1 chat history
GET	/notifications	Authenticated	Fetch active notification feed
GET	/notifications/unread-count	Authenticated	Get count of unread notifications
PUT	/notifications/:id/read	Authenticated	Mark individual notification as read
PUT	/notifications/read-all	Authenticated	Mark all notifications as read
DELETE	/notifications/clear-all	Authenticated	Purge all notifications from mailbox

10.5 Pickup & Matching APIs

Method	Endpoint	Access	Description
POST	/	Volunteer	Schedule a new waste pickup
GET	/my-pickups	Volunteer	Paginated pickup history for volunteer
GET	/available	NGO	Feed of pending pickups matching NGO location
GET	/assigned-to-me	NGO	Pickups currently claimed by the NGO
GET	/:id	Owner / NGO / Admin	Get detailed pickup information
PUT	/:id	Volunteer (Pending)	Modify scheduled pickup details
DELETE	/:id	Volunteer (Pending)	Delete scheduled pickup
PATCH	/:id/cancel	Volunteer (Pending)	Cancel scheduled pickup
PATCH	/:id/status	NGO	Claim, complete, or cancel pickup
PATCH	/:id/reschedule	Volunteer (Missed)	Reschedule missed pickup (max 2 times)
GET	/	Admin	Platform-wide pickup oversight
PUT	/admin/:id	Admin	Administrative pickup data correction
PATCH	/admin/:id/status	Admin	Force pickup status to Completed / Cancelled
DELETE	/admin/:id	Admin	Administrative hard deletion
GET	/v1/matches/opportunities	Owner / NGO / Admin	Skill- and location-based opportunity matches for a volunteer.
GET	/v1/matches/pickups	Owner / NGO / Admin	Location- and waste-type-based pickup matches for an NGO.

10.6 Reports & Data Export

Method	Endpoint	Access	Description
GET	/reports/browse/:type	Volunteer	Paginated JSON preview of personal data
GET	/reports/download/:type	Volunteer	Download file (csv, xlsx, pdf)
GET	/ngo/reports/browse/:type	NGO	Paginated preview of NGO operations
GET	/ngo/reports/download/:type	NGO	Download NGO operational report
GET	/admin/reports/browse/:type	Admin	Paginated preview across entire platform
GET	/admin/reports/:type	Admin	Download platform-wide audit & activity report

10.7 Analytics & Dashboards

Method	Endpoint	Access	Description
GET	/v1/dashboard/metrics	Volunteer / NGO	Personal metrics (pickups, CO_2, drives)
GET	/v1/dashboard/upcoming	Authenticated	Role-scoped upcoming drives & pickups
GET	/v1/stats/leaderboard	Authenticated	Top contributors and caller rank
GET	/v1/stats/recycling-breakdown	Authenticated	Material category weight breakdown
GET	/v1/stats/monthly-trends	Authenticated	Monthly pickup & weight trends
GET	/v1/stats/weekly-trends	Authenticated	Weekly pickup volume distribution
GET	/v1/stats/daily-trends	Authenticated	Daily activity metrics
GET	/v1/stats/co2-factors	Authenticated	Reference table for CO_2 factors
GET	/v1/admin/dashboard/stats	Admin	Platform KPI stats with growth metrics

11. TESTING & VALIDATION

WasteZero's backend APIs were tested primarily using Postman across every milestone, with test coverage extending from basic CRUD correctness to authorization boundaries, validation edge cases, encryption/decryption correctness, and large-dataset streaming behaviour.

11.1 Testing Approach by Module

Authentication & Security

- OTP generation, verification, and expiry handling for registration and password recovery.
- Login success/failure paths, including invalid credentials and unverified accounts.
- Required-field and input validation across all authentication endpoints.

Opportunity & Application

- Successful and invalid CRUD requests, authentication and authorization failures, invalid IDs, and missing fields.
- Search and filter behaviour, ownership restrictions, and application-lifecycle transitions (apply, review, withdraw).

Pickup, Matching, Messaging & Notifications

- Volunteer pickup creation/editing/deletion; NGO accept/reject; admin visibility.
- Socket authentication, message send/receive, encryption and decryption correctness, invalid receiver/empty-message validation, typing indicators, and read receipts.
- Notification creation, real-time delivery, persistence, encryption/decryption, and read-status updates.

Administration, Analytics & Reporting

- RBAC verification across all four roles (anonymous → 401, Volunteer/NGO → 403, Admin → authorized).
- User listing, search, role/suspension filtering, pagination, invalid IDs, and sensitive-field exclusion (passwords, OTPs, JWTs, encryption keys are never returned).
- Suspension/unsuspension, role management (including last-admin protection), and opportunity moderation/restoration, each verified against audit-log creation.
- Dashboard and recycling-analytics correctness (current vs. previous month, growth calculations, category totals).
- Report generation across all four report types and three export formats (CSV, XLSX, PDF), including filtering, validation, and streaming behaviour for large datasets.

11.2 Frontend Integration Testing

Beyond backend API testing, the team performed continuous frontend-backend integration verification at the end of every milestone: reviewing request/response contracts, resolving CORS and routing issues, validating loading/error/empty states across list views, and confirming that role-based route guards correctly restricted access to NGO-only, Volunteer-only, and Admin-only areas of the application.

12. DEPLOYMENT

The WasteZero platform was developed and iteratively deployed to a cloud-hosted environment using Render, allowing the frontend and backend teams to integrate against a live, shared instance throughout each milestone rather than relying solely on local environments.

12.1 Deployment Approach

- The Angular frontend is built into a static production bundle and deployed to Render as a static site, served independently of the API layer.
- The Node.js/Express.js backend is deployed as a standalone Render web service, exposing the versioned REST API and the Socket.IO real-time layer.
- MongoDB is provisioned as a managed, cloud-hosted database via MongoDB Atlas, decoupled from the application server.
- Environment configuration (API base URLs, secrets, database connection strings) is externalised via environment variables configured within Render's service settings and Angular's environment files, rather than hard-coded, allowing the same codebase to run in development and production contexts.

12.2 Environment Configuration

The frontend uses Angular's environment configuration files to target the correct Render-hosted backend API URL at build time, avoiding hard-coded endpoints and simplifying the transition between local development and the shared integration environment used throughout the internship. The backend connects to MongoDB Atlas via a connection string injected as a Render environment variable, keeping credentials out of source control.

12.3 Release Cadence

Each milestone concluded with a stabilisation and integration pass — reviewing pending features, resolving frontend-backend integration issues, and performing a final merge — before the milestone's deliverables were considered complete and demonstrable against the live Render deployment.

13. CHALLENGES & SOLUTIONS

Challenge	Solution
CORS and authentication-integration failures between the Angular frontend and the Express backend during early Milestone 1 development.	Systematic debugging of request/response headers and CORS configuration; authentication flow and routing issues were identified and resolved before the Milestone 1 submission.
A team member's departure left the Change Password feature without a frontend owner.	Dhruv took over ownership of the Change Password UI and its backend integration, ensuring no loss of milestone functionality.
The initial Opportunity module needed to evolve from a basic scaffold into a fully-featured module (search, filtering, pagination, validation) without disrupting what was already integrated.	The module was intentionally built in two passes: Aruneshwar established the architectural foundation, after which Dhruv extended it feature-by-feature, reviewing and refining the initial implementation before final integration.
Coordinating three parallel backend workstreams (pickup management, matching, messaging/notifications) against a single Angular frontend within a two-week milestone.	Backend responsibilities were clearly partitioned (Vaishnavi: Socket.IO/messaging/notifications; Apoorva J: pickups/matching/REST layer), with a defined API contract so the frontend could integrate against a stable interface.
Storing chat and notification content securely rather than as plaintext in MongoDB.	AES-256-GCM encryption was implemented for both messages and notifications at the database layer, with transparent decryption on authorised retrieval.
Preventing duplicate conversations when two users could message each other from either direction.	A deterministic conversation-ID generator sorts the two participant IDs before hashing, guaranteeing a single canonical conversation per user pair.
Generating large administrative reports (Users, Pickups, Opportunities, Full Activity) without exhausting server memory.	CSV, Excel, and PDF exporters were built around a streaming pipeline — MongoDB cursor → transform stream → output stream — rather than loading the full dataset into memory.
Preventing unsafe administrative actions such as an admin demoting themselves or removing the last remaining admin account.	Explicit self-action and last-admin safeguards were added to the role-management endpoint, alongside mandatory audit logging of every change.
Ensuring suspended users could not continue using real-time features after an administrator suspended their account.	Suspension was treated as a security event: it actively disconnects the user's live Socket.IO session in addition to blocking further REST API access.

14. TEAM MEMBERS & CONTRIBUTIONS

WasteZero was built by a four-person team spanning frontend and backend development. The table below summarises each member's primary role and area of ownership, followed by a detailed, milestone-by-milestone account of individual contributions.

Team Member	Primary Role	Core Ownership
Vaishnavi Shrivastava	Team Lead & Backend Developer	Authentication, Opportunity backend, real-time messaging & notifications, platform governance and audit systems
Apoorva J	Backend Developer	OTP/security, Application management, pickup & matching logic, analytics and reporting backend
Dhruv	Frontend Developer	Authentication UI, dashboards, profile, opportunities, pickups, messaging, notifications, and admin frontend
Aruneshwar	Frontend Developer	Initial Opportunity module foundation; Administration & Reporting frontend

14.1 Vaishnavi Shrivastava - Team Lead & Backend Developer

Vaishnavi served as the backend lead for WasteZero, responsible for designing, implementing, testing, and integrating major backend components across all four milestones — spanning authentication, opportunity management, real-time messaging, security, administration, moderation, and audit logging.

Milestone 1

- Developed the complete Authentication Controller and User Controller.
- Implemented Registration, Login, Change Password, and Update Profile APIs, with authentication/user-management routing.
- Integrated MongoDB via Mongoose; implemented password hashing and JWT-based protected API access.
- Supported the frontend team in integrating authentication and profile APIs.

Milestone 2

- Designed the Opportunity schema and finalised the Opportunity API contract for the team.
- Implemented the complete Opportunity CRUD, My Opportunities, Search, and Filter APIs.
- Implemented role-based authorization and ownership middleware, and the reusable apiResponse.js response-standardisation utility.
- Tested all major Opportunity APIs via Postman and supported frontend integration.

Milestone 3

- Designed and implemented the Socket.IO real-time messaging architecture (message:send/new/typing/read).
- Implemented AES-256-GCM encryption for messages and notifications, and deterministic conversation-ID generation.
- Implemented authenticated per-user Socket.IO rooms and the Notification Service (dispatch, listForUser, markRead).
- Implemented input validation and the full read-receipt workflow; tested messaging and notification flows extensively.

Milestone 4

- Owned platform governance: admin user listing/search/filtering, suspension/unsuspension with session/socket revocation, and role management with last-admin protection.
- Implemented opportunity moderation and restoration, the requireAdmin RBAC middleware, and suspended-user security enforcement.
- Built the append-only Administrative Audit Log system and its API, plus administrative rate limiting and validation.
- Worked closely with the frontend team throughout the project on API contracts, integration troubleshooting, and Postman-based verification.

14.2 Apoorva J - Backend Developer

Apoorva contributed primarily to the backend development of Authentication security, Application Management, Pickup & Matching, and — in the final milestone — Analytics, Dashboard, and Reporting.

Milestone 1

- Implemented the Registration OTP and Email Verification APIs, and the Verify OTP functionality.
- Implemented the Forgot Password and Reset Password APIs with OTP-gated verification.
- Contributed to the OTP-verification flow for Change Password (with Vaishnavi) and to authentication-related input validation.

Milestone 2

- Designed the Application schema modelling the Volunteer–Opportunity–Application relationship.
- Implemented the complete Application API: apply, get applications, get by ID, update status, withdraw, and My Applications.
- Built the shared query-builder/pagination utility used across list-based APIs, and tested the module via Postman.

Milestone 3

- Implemented the complete Pickup Management backend — volunteer create/edit/delete, NGO accept/reject, and admin visibility.
- Implemented the skill-and-location Opportunity Matching algorithm and the location-and-waste-type Pickup Matching algorithm.
- Contributed to the REST API layer supporting the Messaging and Notification modules.

Milestone 4

- Implemented Admin Dashboard Analytics and the User Personal Dashboard endpoints, including growth-percentage calculations.
- Implemented the Recycling Breakdown analytics endpoint and WasteStats creation/update logic, using MongoDB aggregation pipelines.
- Built the CO_2-calculator utility, the administrative Pickup Analytics endpoint, and the Pickup Status Override endpoint.
- Delivered the complete Report Generation System — Users, Pickups, Opportunities, and Full Activity reports — with CSV, Excel/XLSX, and PDF exporters, streaming architecture, filtering/validation, rate limiting, and audit integration.

14.3 Dhruv - Frontend UI, Dashboard & Integration Developer

Dhruv made a major, sustained contribution to the WasteZero frontend across all four milestones, covering authentication, dashboards, profile management, opportunity management, applications, pickup management, matching, messaging, notifications, UI/UX improvements, and the final administration and reporting interface.

Milestone 1

- Implemented and integrated the complete Login, Registration, and OTP frontend flows with backend APIs; resolved CORS and routing/integration issues.
- Took over and completed the Change Password UI and its backend integration after a team change.
- Designed and developed the Role-Based Dashboard UI and the Profile Management (Update Profile) interface.

Milestone 2

- Reviewed and substantially extended the initial Opportunity module — architecture, routing, and live backend integration.
- Delivered the finalised Opportunity List (search, filter, pagination, loading/error/empty states), the shared Create/Edit form, Opportunity Detail page, and Delete Confirmation dialog.
- Built the My Opportunities dashboard and the complete Applications module (apply, My Applications, status badges), plus NGO/Admin route guards.
- Performed the final Milestone 2 frontend integration and merge with the finalised backend APIs.

Milestone 3

- Implemented Schedule Pickup and Pickup History for volunteers; Available Pickups and My Assigned Pickups for NGOs; and Admin Pickup Management.
- Integrated Opportunity Matching results into the volunteer dashboard, with matching notifications.
- Built the complete messaging frontend — user/conversation search, chat history, typing indicators, sent/read status — plus direct chat entry points from opportunities and applications.
- Implemented the categorised General/Text notification system with unread indicators, covering both matching and messaging notifications.

Milestone 4

- Completed and refined the Admin Dashboard and the Admin Controls Panel frontend.
- Delivered the Reports/Dashboard reporting interface and performed final refinement/integration of the administrative frontend.
- Contributed sustained UI/UX enhancement work across the application — responsiveness, consistency, and a more modern, professional frontend experience.

14.4 Aruneshwar - Frontend Developer

Aruneshwar established the initial frontend foundation for the Opportunity Management module in Milestone 3 and delivered the Administration & Reporting frontend in Milestone 4.

Milestone 3 - Initial Opportunity Module

- Implemented the initial Opportunity List and Opportunity Detail pages, with navigation between them.
- Created the initial Opportunity Service structure with basic CRUD methods for backend communication.
- Configured the initial routing structure and component/service architecture for the Opportunity module, preparing it for subsequent extension by Dhruv.

Milestone 4 - Administration & Reporting Frontend

- Developed the Admin Users page: users table, search, role/status filtering, pagination, suspend/activate actions with confirmation modals, and loading/error/empty states.
- Implemented the Admin Logs page with action- and date-based filtering, pagination, and structured presentation of administrative activity.
- Built the Reports interface supporting User, Pickup, Opportunity, and Full Activity reports with report-type selection, date filtering, CSV/Excel/PDF format selection, and download handling; used temporary mock data where backend report APIs were still in progress.

Implemented the /admin/users, /admin/logs, and /admin/reports navigation routes and frontend-level route protection restricting the Administration section to Admin users.

15. FUTURE ENHANCEMENTS

- Native mobile applications (iOS/Android) built on top of the existing REST and Socket.IO APIs.
- Map-based visualisation of pickup requests and NGO service areas to complement text-based location matching.
- Push notifications (web and mobile) in addition to the current in-app notification system.
- A gamification / rewards layer recognising high-impact volunteers based on accumulated CO_2-saved and pickup-completion metrics.
- Expanded analytics — predictive matching suggestions and trend forecasting built on top of the existing MongoDB aggregation pipelines.
- Integration with third-party logistics or municipal waste-management systems for large-scale pickup coordination.
- Automated end-to-end and unit test suites to complement the existing Postman-based manual API testing.
- Multi-language support for the Angular frontend to broaden platform accessibility.

16. CONCLUSION

The WasteZero internship project successfully delivered a complete, full-stack Smart Waste Pickup & Recycling Platform across four structured milestones. Beginning with a secure, role-based authentication foundation, the team progressively built out opportunity management, intelligent skill- and location-based matching, encrypted real-time communication, and a comprehensive administration and reporting layer — culminating in a platform capable of supporting Volunteers, NGOs, and Administrators through their respective end-to-end workflows.

The project demonstrated the team's ability to design and implement a layered, secure backend architecture (Node.js, Express.js, MongoDB, Socket.IO, JWT, and AES-256-GCM encryption); build a responsive, role-aware Angular frontend integrated continuously against evolving backend APIs; and coordinate effectively across four contributors with clearly partitioned, complementary areas of ownership. Rigorous Postman-based API testing, careful authorization boundary enforcement, and an append-only audit-logging system reflect a security- and quality-conscious engineering approach throughout.

Beyond the functional deliverables, the internship provided the team with practical, end-to-end experience in the full software development lifecycle — requirements analysis, schema and API design, iterative implementation, integration, testing, and documentation — culminating in a platform that meaningfully supports responsible, community-driven waste management.

17. REFERENCES

- WasteZero Project Statement and Milestone Specification — internal project brief (Milestones 1–4).
- Angular Documentation — https://angular.dev
- Node.js Documentation — https://nodejs.org/docs
- Express.js Documentation — https://expressjs.com
- MongoDB & Mongoose Documentation — https://www.mongodb.com/docs and https://mongoosejs.com
- Socket.IO Documentation — https://socket.io/docs
- JSON Web Tokens (JWT) — https://jwt.io/introduction
- Node.js Crypto Module (AES-256-GCM) — https://nodejs.org/api/crypto.html
- PDFKit Documentation — https://pdfkit.org
- ExcelJS Documentation — https://github.com/exceljs/exceljs
- Postman API Testing Platform — https://www.postman.com